COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 9 EXERCISES
STRUCTURED QUERY LANGUAGE (SQL): DATA MANIPULATION LANGUAGE (DML)

BOATRESERVATIONS RELATIONAL DATABASE

Sailor(sailorId, sName, hkid, rating, age)

Reserves(*sailorId, boatId*, rDate)

Boat(boatId, bName, color, bType)

Attribute names in italics are foreign key attributes.

Sailor

sailorId	sName	hkid	rating	age
10	Emily	P936219(3)	null	16
15	Zoey	K091876(5)	null	17
22	Dustin	A298456(6)	7	45
29	Brutus	D746239(0)	1	33
31	Lucy	P930281(5)	10	55
32	Andy	M028391(4)	8	25
58	Rachael	A819204(3)	10	17
64	Horatio	F710943(2)	7	35
71	Zorba	P910562(4)	1	16
74	Horatio	T810463(1)	9	35
85	Amy	B719037(8	3	25
95	Bob	E012735(5)	3	63
99	Chris	F710943(2)	10	30

13 tuples

Reserves

sailorId	*boatId*	rDate
22	103	08-SEP-25
22	104	10-OCT-25
22	101	10-OCT-25
22	102	10-NOV-25
31	102	10--OCT-25
31	103	06--NOV-25
31	102	12-NOV-25
64	101	05-SEP-25
64	102	08-SEP-25
74	103	08-SEP-25
99	102	08-AUG-25

11 tuples

Boat

boatId	bName	color	bType
101	Interlake	blue	dinghy
102	Interlake	red	sloop
103	Clipper	green	dinghy
104	Marine	red	catamaran
105	Serenity	cyan	null

5 tuples

EXERCISE 1

☞ (Marine, 1)
(Interlake, 5)

Find the number of reservations made for each red boat that has a reservation. Include in the query result tuples attributes boat name and number of reservations. Order the query result tuples by number of reservations ascending.

Is this a correct solution? **Illegal!!!** Why?

```
select bName, count(*) as numReservations
from Boat natural join Reserves
where color='red'
group by boatId
order by numReservations;
```

☞ **All non-aggregate attributes in the select clause must appear in the group by clause**
(i.e., bName must appear in the group by clause).

Why is this necessary?

☞ **To ensure that there is at most one value to output for each group.**

Consider this example.

```
select bName, count(*) as colorCount
from Boat
group by color;
```

For each color there could be many boats; which bName value to output?

EXERCISE 1 (CONTD)

☞ (Marine, 1)
(Interlake, 5)

Find the number of reservations made for each red boat that has a reservation. Include in the query result tuples attributes boat name and number of reservations. Order the query result tuples by number of reservations ascending.

```
select bName, count(*) as numReservations
from Boat natural join Reserves
where color='red'
group by bName, boatId
order by numReservations;
```

sailorId	boatId	rDate	bName	color	bType
22	102	10-OCT-25	Interlake	red	sloop
22	104	10-OCT-25	Marine	red	catamaran
31	102	12-NOV-25	Interlake	red	sloop
31	102	12-NOV-25	Interlake	red	sloop
64	102	08-SEP-25	Interlake	red	sloop
99	102	08-AUG-25	Interlake	red	sloop

Get the reservations for red boats.

a group

a group

Group by bName and boatId.

Query result

bName	numReservations
Marine	1
Interlake	5

Project on bName and numReservations for each group and **order by** numReservations.

EXERCISE 1 (CONTD)

☞ (Marine, 1)
(Interlake, 5)

Find the number of reservations made for each red boat that has a reservation. Include in the query result tuples attributes boat name and number of reservations. Order the query result tuples by number of reservations ascending.

Is it really necessary to group by boatId?

```
select bName, count(*) as numReservations
from Boat natural join Reserves
where color='red'
group by bName
order by numReservations;
```

This looks OK, but do you see any problem with this solution?
(See next slide.)

sailorId	boatId	rDate	bName	color	bType
22	102	10-OCT-25	Interlake	red	sloop
22	104	10-OCT-25	Marine	red	catamaran
31	102	12-NOV-25	Interlake	red	sloop
31	102	12-NOV-25	Interlake	red	sloop
64	102	08-SEP-25	Interlake	red	sloop
99	102	08-AUG-25	Interlake	red	sloop

Get the reservations for red boats.

a group

a group

Group by bName.

Query result

bName	numReservations
Marine	1
Interlake	5

Project on bName and numReservations for each group and **order by** numReservations.

EXERCISE 1 (CONT'D)

☞ (Marine, 1)
(Interlake, 5)

Find the number of reservations made for each boat that has a reservation. Include in the query result tuples attributes boat name and number of reservations. Order the query result tuples by number of reservations ascending.

Suppose we change the query to the above.

What is the result?

```
select bName, count(*) as numReservations
from Boat natural join Reserves
group by bName
order by numReservations;
```

Since bName is not unique, grouping on it can get an incorrect result!

sailorId	boatId	rDate	bName	color	bType
22	101	10-OCT-25	Interlake	blue	dinghy
64	101	05-SEP-25	Interlake	blue	dinghy
22	102	10-OCT-25	Interlake	red	sloop
31	102	12-NOV-25	Interlake	red	sloop
31	102	12-NOV-25	Interlake	red	sloop
64	102	08-SEP-25	Interlake	red	sloop
99	102	08-AUG-25	Interlake	red	sloop
22	103	08-SEP-25	Clipper	green	dinghy
31	103	06-NOV-25	Clipper	green	dinghy
74	103	08-SEP-25	Clipper	green	dinghy
22	104	10-OCT-25	Marine	red	catamaran

a group

a group

a group

Join Boat and Reserves to get the reservations for all boats.

Group by bName.

Query result

bName	numReservations
Marine	1
Clipper	3
Interlake	7

Project on bName and numReservations for each group and **order by** numReservations.

EXERCISE 1 (CONTD)

☞ (Marine, 1)
(Interlake, 5)

Find the number of reservations made for each boat that has a reservation. Include in the query result tuples attributes boat name and number of reservations. Order the query result tuples by number of reservations ascending.

Correct solution. Include boatId in the **group by** clause.

```
select bName, count(*) as numReservations
from Boat natural join Reserves
group by bName, boatId
order by numReservations;
```

Recall attributes in the **group by** clause do not have to appear in the **select** clause.

sailorId	boatId	rDate	bName	color	bType
22	101	10-OCT-25	Interlake	blue	dinghy
64	101	05-SEP-25	Interlake	blue	dinghy
22	102	10-OCT-25	Interlake	red	sloop
31	102	12-NOV-25	Interlake	red	sloop
31	102	12-NOV-25	Interlake	red	sloop
64	102	08-SEP-25	Interlake	red	sloop
99	10	08-AUG-25	Interlake	red	sloop
22	103	08-SEP-25	Clipper	green	dinghy
31	103	06-NOV-25	Clipper	green	dinghy
74	103	08-SEP-25	Clipper	green	dinghy
22	104	10-OCT-25	Marine	red	catamaran

a group

a group

a group

a group

Join Boat and Reserves to get the reservations for all boats.

Group by bName and boatId.

Query result

bName	numReservations
Marine	1
Interlake	2
Clipper	3
Interlake	5

Project on bName and numReservations for each group and **order by** numReservations.

EXERCISE 2

Find the number of reservations made by each sailor.
Include in the query result tuples attributes sailor name and number of reservations.
Order the query result tuples first by number of reservations ascending and then by name ascending.

☞ (Amy, 0) (Andy, 0) (Bob, 0) (Brutus, 0) (Emily, 0) (Rachael, 0) (Zoey, 0)
(Zorba, 0) (Chris, 1) (Horatio, 1) (Horatio, 2) (Lucy, 3) (Dustin, 4)

```
select sName, count(*) as numReservations
from Sailor natural join Reserves
group by sName
order by numReservations, sName;
```

The count for Horatio is misleading.

Not all sailors are included.

Query result

sName	numReservations
Chris	1
Horatio	3
Lucy	3
Dustin	4

How about grouping by sailorId and sName?

```
select sName, count(*) as numReservations
from Sailor natural join Reserves
group by sailorId, sName
order by numReservations, sName;
```

This fixes the count but still all sailors are not included.

What's the problem?

Query result

sailorId	numReservations
Chris	1
Horatio	1
Horatio	2
Lucy	3
Dustin	4

EXERCISE 2 (CONT'D)

Find the number of reservations made by each sailor.
Include in the query result tuples attributes sailor name and number of reservations.
Order the query result tuples first by number of reservations ascending and then by name ascending.

sailorId	sName	hkid	rating	age	boatId	rDate
22	Dustin	A298456(6)	7	45	101	10-OCT-25
22	Dustin	A298456(6)	7	45	102	10-NOV-25
22	Dustin	A298456(6)	7	45	103	08-SEP-25
22	Dustin	A298456(6)	7	45	104	10-OCT-25
31	Lucy	P930281(5)	10	55	102	10-OCT-25
31	Lucy	P930281(5)	10	55	103	06-NOV-25
31	Lucy	P930281(5)	10	55	102	12-NOV-25
64	Horatio	F710943(2)	7	35	101	05-SEP-25
64	Horatio	F710943(2)	7	35	102	08-SEP-25
74	Horatio	T810463(1)	9	35	103	08-SEP-25
99	Chris	H814243(7)	10	30	102	08-AUG-25
10	Emily	P936219(3)	null	16		
15	Zoey	K091876(5)	null	17		
29	Brutus	D746239(0)	1	33		
32	Andy	M028391(4)	8	25		
58	Rachael	A819204(3)	10	17		
71	Zorba	P910562(4)	1	16		
85	Amy	B719037(8)	3	25		
95	Bob	E012735(5)	3	63		

natural join result

```
select sName, count(*) as numReservations
from Sailor natural join Reserves
group by sailorId, sName
order by numReservations, sName;
```

Query result

sailorId	numReservations
Chris	1
Horatio	1
Horatio	2
Lucy	3
Dustin	4

☞ Some Sailor tuples have no match in the Reserves relation.

How to deal with this problem?

☞ Use outer join.

EXERCISE 2 (CONT'D)

Find the number of reservations made by each sailor.
Include in the query result tuples attributes sailor name and number of reservations.
Order the query result tuples first by number of reservations ascending and then by name ascending.

```
select sName, count(boatId) as numReservations
from Sailor natural left outer join Reserves
group by sailorId, sName
order by numReservations, sName;
```

Recall left outer join keeps all copies of the common attributes; natural left outer join keeps only one copy of the common attributes.

Recall The count of an empty collection is 0. The count of null is also 0.

Is this a correct solution?

No!

Why?

```
select sName, count(sailorId) as numReservations
from Sailor natural left outer join Reserves
group by sailorId, sName
order by numReservations, sName;
```

Counting is done on the sailor ids and all of them appear at least once in the query result tuples attributes.

Sailor natural left outer join Reserves

sailorId	sName	hkid	rating	age	boatId	rDate
22	Dustin	A298456(6)	7	45	101	10-OCT-25
22	Dustin	A298456(6)	7	45	102	10-NOV-25
22	Dustin	A298456(6)	7	45	103	08-SEP-25
22	Dustin	A298456(6)	7	45	104	10-OCT-25
31	Lucy	P930281(5)	10	55	102	10-OCT-25
31	Lucy	P930281(5)	10	55	103	06-NOV-25
31	Lucy	P930281(5)	10	55	102	12-NOV-25
64	Horatio	F710943(2)	7	35	101	05-SEP-25
64	Horatio	F710943(2)	7	35	102	08-SEP-25
74	Horatio	T810463(1)	9	35	103	08-SEP-25
99	Chris	H814243(7)	10	30	102	08-AUG-25
10	Emily	P936219(3)	null	16	null	null
15	Zoey	K091876(5)	null	17	null	null
29	Brutus	D746239(0)	1	33	null	null
32	Andy	M028391(4)	8	25	null	null
58	Rachael	A819204(3)	10	17	null	null
71	Zorba	P910562(4)	1	16	null	null
85	Amy	B719037(8)	3	25	null	null
95	Bob	E012735(5)	3	63	null	null

☞ Can also count on rDate.

EXERCISE 3

Find all sailors who have the highest rating.
Include in the query result tuples attributes sailor id, name, rating and age.
Order the query result tuples by age ascending.

☞ (58, Rachael, 10, 17) (99, Chris, 10, 30) (31, Lucy, 10, 55)

Is this a correct solution?

No! Why?

```
select sailorId, sName, rating, age
from Sailor
where rating=max(rating)
order by age;
```

Illegal SQL!
There is no **max**(rating) value to compare in the **where** clause.
☞ The max rating value must be computed by a query!

Is this a correct solution?

No! Why?

```
select sailorId, sName, rating, age, max(rating)
from Sailor
order by age;
```

Illegal SQL!
A query that returns multiple tuples cannot contain an aggregate function in the **select** clause.
☞ There are multiple tuples in the result, but only one **max**(rating) value!

Sailor(sailorId, sName, hkid, rating, age)

EXERCISE 3 (CONTD)

☞ (58, Rachael, 10, 17)
(99, Chris, 10, 30)
(31, Lucy, 10, 55)

Find all sailors who have the highest rating.
Include in the query result tuples attributes sailor id, name, rating and age.
Order the query result tuples by age ascending.

```
select sailorId, sName, rating, age
from Sailor
where rating=(select max(rating)
              from Sailor)
order by age;
```

max(rating)
10

Use a (sub)query to compute the maximum (i.e., highest) rating.

sailorId	sName	hkid	rating	age
10	Emily	P936219(3)	null	16
15	Zoey	P936219(3)	null	17
22	Dustin	A298456(6)	7	45
29	Brutus	D746239(0)	1	33
31	Lucy	P930281(5)	10	55
32	Andy	M028391(4)	8	25
58	Rachael	A819204(3)	10	17
64	Horatio	F710943(2)	7	35
71	Zorba	P910562(4)	1	16
74	Horatio	T810463(1)	9	35
85	Amy	B719037(8)	3	25
95	Bob	E012735(5)	3	63
99	Chris	H814243(7)	10	30

sailorId	sName	hkid	rating	age
31	Lucy	P930281(5)	10	55
58	Rachael	A819204(3)	10	17
99	Chris	H814243(7)	10	30

Use the computed maximum rating value to get the sailor tuples with the maximum (i.e., highest) rating.

Query result

sailorId	sName	rating	age
58	Rachael	10	17
99	Chris	10	30
31	Lucy	10	55

Project on sailorId, sName, rating and age and **order by** age.

Recall
By default, null is the largest value.

EXERCISE 3 (CONTD)

☞ (58, Rachael, 10, 17)
(99, Chris, 10, 30)
(31, Lucy, 10, 55)

Find all sailors who have the highest rating.
Include in the query result tuples attributes sailor id, name, rating and age.
Order the query result tuples by age ascending.

```
select sailorId, sName, rating, age
from Sailor
order by rating desc nulls last
fetch first 1 row with ties;
```

Is this a correct solution?

Yes, but … cannot order the result by age.

Get all sailors ordered by rating desc.

sailorId	sName	hkid	rating	age
31	Lucy	P930281(5)	10	55
58	Rachael	A819204(3)	10	17
99	Chris	H814243(7)	10	30
74	Horatio	T810463(1)	9	35
32	Andy	M028391(4)	8	25
22	Dustin	A298456(6)	7	45
64	Horatio	F710943(2)	7	35
85	Amy	B719037(8)	3	25
95	Bob	E012735(5)	3	63
29	Brutus	D746239(0)	1	33
71	Zorba	P910562(4)	1	16
10	Emily	P936219(3)	null	16
15	Zoey	P936219(3)	null	17

sailorId	sName	hkid	rating	age
31	Lucy	P930281(5)	10	55
58	Rachael	A819204(3)	10	17
99	Chris	H814243(7)	10	30

Retain only the sailor tuples with the maximum rating.

Query result

sailorId	sName	rating	age
31	Lucy	10	55
58	Rachael	10	17
99	Chris	10	30

Project on sailorId, sName, rating and age .

Use set membership

EXERCISE 3 (CONTD)

☞ (58, Rachael, 10, 17)
(99, Chris, 10, 30)
(31, Lucy, 10, 55)

Find all sailors who have the highest rating.
Include in the query result tuples attributes sailor id, name, rating and age.
Order the query result tuples by age ascending.

```
select sailorId, sName, rating, age
from Sailor
where rating >=all (select rating
                    from Sailor
                    where rating is not null)
order by age;
```

rating
7
1
8
8
10
7
10
9
3
3
10

Select all non-null ratings.

sailorId	sName	hkid	rating	age
10	Emily	P936219(3)	null	16
15	Zoey	P936219(3)	null	17
22	Dustin	A298456(6)	7	45
29	Brutus	D746239(0)	1	33
31	Lucy	P930281(5)	10	55
32	Andy	M028391(4)	8	25
58	Rachael	A819204(3)	10	17
64	Horatio	F710943(2)	7	35
71	Zorba	P910562(4)	1	16
74	Horatio	T810463(1)	9	35
85	Amy	B719037(8)	3	25
95	Bob	E012735(5)	3	63
99	Chris	H814243(7)	10	30

sailorId	sName	hkid	rating	age
31	Lucy	P930281(5)	10	55
58	Rachael	A819204(3)	10	17
99	Chris	H814243(7)	10	30

Retain only the sailor tuples with the maximum rating.

Query result

sailorId	sName	rating	age
58	Rachael	10	17
99	Chris	10	30
31	Lucy	10	55

Project on sailorId, sName, rating and age and **order by** age.

EXERCISE 3 (CONTD)

What is the result if we replace ">=all" with ">all"?

```
select sailorId, sName, rating, age
from Sailor
where rating>all (select rating
                  from Sailor
                  where rating is not null)
order by age;
```

sailorId	sName	hkid	rating	age
10	Emily	P936219(3)	null	16
15	Zoey	P936219(3)	null	17
22	Dustin	A298456(6)	7	45
29	Brutus	D746239(0)	1	33
31	Lucy	P930281(5)	10	55
32	Andy	M028391(4)	8	25
58	Rachael	A819204(3)	10	17
64	Horatio	F710943(2)	7	35
71	Zorba	P910562(4)	1	16
74	Horatio	T810463(1)	9	35
85	Amy	B719037(8)	3	25
95	Bob	E012735(5)	3	63
99	Chris	H814243(7)	10	30

Query result:
No sailors are selected.

Why?

No ratings are greater than the maximum rating!

☞ Recall ">**all**" is equivalent to greater than the maximum.

rating
7
1
8
8
10
7
10
9
3
3
10

Select all non-null ratings.

EXERCISE 3 (CONTD)

What is the result if we replace ">=all" with ">=some"?

```
select sailorId, sName, rating, age
from Sailor
where rating>=some (select rating
                    from Sailor
                    where rating is not null)
order by age;
```

sailorId	sName	hkid	rating	age
10	Emily	P936219(3)	null	16
15	Zoey	P936219(3)	null	17
22	Dustin	A298456(6)	7	45
29	Brutus	D746239(0)	1	33
31	Lucy	P930281(5)	10	55
32	Andy	M028391(4)	8	25
58	Rachael	A819204(3)	10	17
64	Horatio	F710943(2)	7	35
71	Zorba	P910562(4)	1	16
74	Horatio	T810463(1)	9	35
85	Amy	B719037(8)	3	25
95	Bob	E012735(5)	3	63
99	Chris	H814243(7)	10	30

Query result:
All sailors with ratings are selected.

Why?

All ratings are greater than or equal to the minimum rating!

☞ Recall ">**some**" is equivalent to greater than the minimum.

rating
7
1
8
8
10
7
10
9
3
3
10

Select all non-null ratings.

Use *only* set membership

EXERCISE 4

☞ (Chris) (Dustin) (Horatio) (Lucy)

Find the sailors who have reserved a red boat.
Include in the query result tuples attributes only unique sailor names.
Order the query result tuples by name ascending.

```
select sName
from Sailor
where sailorId in (select distinct sailorId
                   from Reserves
                   where boatId in (select boatId
                                    from Boat
                                    where color='red'))
order by sName
```

Query result

sName
Chris
Dustin
Horatio
Lucy

Select the names of sailors who have reserved a red boat.

sailorId
22
31
64
99

Select the distinct ids of sailors who have reserved a red boat.

boatId
102
104

Select the ids of red boats.

Is it necessary to specify **distinct** in the second **select** clause?

No! Why?

Each tuple of the outermost **select** matches only once regardless of how many duplicates there are in the result of the middle **select**.

EXERCISE 4 (CONT'D)

What if we replace the first **in** with **not in**?

Stated in words, what does this result represent?

```
select sName
from Sailor
where sailorId not in (select distinct sailorId
                       from Reserves
                       where boatId in (select boatId
                                        from Boat
                                        where color='red'))
order by sName;
```

Query result

sName
Amy
Andy
Bob
Brutus
Emily
Horatio
Rachael
Zoey
Zorba

Select the names of sailors who have not reserved a red boat (including reserved no boat).

sailorId
22
31
64
99

Select the distinct ids of sailors who have reserved a red boat.

boatId
102
104

Select the ids of red boats.

Sailor(sailorId, sName, hkid, rating, age)

Reserves(*sailorId, boatId,* rDate)

Boat(boatId, bName, color, bType)

EXERCISE 4 (CONT'D)

What if we replace the second **in** with **not in**?

```
select sName
from Sailor
where sailorId in (select distinct sailorId
                   from Reserves
                   where boatId not in (select boatId
                                        from Boat
                                        where color='red'))
order by sName;
```

Query result

sName
Dustin
Horatio
Horatio
Lucy

Select the names of sailors who have reserved a boat other than a red boat (excludes sailors who have not reserved any boat).

sailorId
22
31
64
74

Select the distinct ids of sailors who have reserved a boat other than a red boat.

Stated in words, what does this result represent?

boatId
102
104

Select the ids of red boats.

Sailor(sailorId, sName, hkid, rating, age)

Reserves(*sailorId*, *boatId*, rDate)

Boat(boatId, bName, color, bType)

EXERCISE 4 (CONT'D)

What if we replace both in's with not in?

Stated in words, what does this result represent?

```
select sName
from Sailor
where sailorId not in (select distinct sailorId
                       from Reserves
                       where boatId not in (select boatId
                                            from Boat
                                            where color='red'))
order by sName;
```

Query result

sName
Amy
Andy
Bob
Brutus
Chris
Emily
Rachael
Zoey
Zorba

Select the names of sailors who have reserved only a red boat (i.e., Chris) or have reserved no boat.

sailorId
22
31
64
74

Select the distinct ids of sailors who have reserved a boat other than a red boat.

boatId
102
104

Select the ids of red boats.

Sailor(sailorId, sName, hkid, rating, age)

Reserves(*sailorId, boatId,* rDate)

Boat(boatId, bName, color, bType)

Use **exists**

EXERCISE 4 (CONT'D)

☞ (Chris) (Dustin) (Horatio) (Lucy)

Find the sailors who have reserved a red boat.
Include in the query result tuples attributes only unique sailor names.
Order the query result tuples by name ascending.

```
select sName
from Sailor S
where exists (select *
                from Reserves natural join Boat
                where Reserves.sailorId=S.sailorId
                  and color='red')
order by sName;
```

sailorId	sName
22	Dustin
29	Brutus
31	Lucy
32	Andy
58	Rachael
64	Horatio
71	Zorba
74	Horatio
85	Amy
95	Bob
99	Chris

Reserves **natural join** Boat **where** color='red'

boatId	sailorId	rDate	bName	color	bType
102	22	10-NOV-25	Interlake	red	sloop
102	64	08-SEP-25	Interlake	red	sloop
102	31	10-OCT-25	Interlake	red	sloop
104	22	10-OCT-25	Marine	red	catamaran
102	99	08-AUG-25	Interlake	red	sloop
102	31	12-NOV-25	Interlake	red	sloop

Sailor(sailorId, sName, hkid, rating, age)
Reserves(*sailorId, boatId*, rDate)
Boat(boatId, bName, color, bType)

EXERCISE 4 (CONT'D)

☞ (Chris) (Dustin)
(Horatio) (Lucy)

Find the sailors who have reserved a red boat.
Include in the query result tuples attributes only unique sailor names.
Order the query result tuples by name ascending.

```
with RedBoatReservations (sailorId) as
(select sailorId
 from Reserves natural join Boat
 where color='red')
select distinct sName
from Sailor natural join RedBoatReservations
order by sName;
```

sailorId	sName
22	Dustin
29	Brutus
31	Lucy
32	Andy
58	Rachael
64	Horatio
71	Zorba
74	Horatio
85	Amy
95	Bob
99	Chris

RedBoatReservations

sailorId
22
64
31
22
99
31

Sailor(sailorId, sName, hkid, rating, age)
Reserves(*sailorId, boatId*, rDate)
Boat(boatId, bName, color, bType)

EXERCISE 5

Find the sailors for which the average rating of their age is equal to the maximum average rating of all ages.
Include in the query result tuples attributes maximum average rating and age.
Order the query result tuples by age ascending.

Sailor

sailorId	sName	hkid	rating	age
10	Emily	P936219(3)	null	16
15	Zoey	K091876(5)	null	17
22	Dustin	A298456(6)	7	45
29	Brutus	D746239(0)	1	33
31	Lucy	P930281(5)	10	55
32	Andy	M028391(4)	8	25
58	Rachael	A819204(3)	10	17
64	Horatio	F710943(2)	7	35
71	Zorba	P910562(4)	1	16
74	Horatio	T810463(1)	9	35
85	Amy	B719037(8	3	25
95	Bob	E012735(5)	3	63
99	Chris	F710943(2)	10	30

13 tuples

1. Find the average rating for each age.
2. Find the maximum overall average rating for all ages.
3. For the result of 1, find those tuples where the average rating is equal to the result of 2 (i.e., the average rating for an age is equal to the maximum average rating of all ages).

EXERCISE 5

Find the sailors for which the average rating of their age is equal to the maximum average rating of all ages.
Include in the query result tuples attributes maximum average rating and age.
Order the query result tuples by age ascending.

☞ (10, 17) (10, 30) (10, 55)

Is this a correct solution?

No! Why?

```
select avg(rating) as maxAvgRating, age
from Sailor
where avg(rating)=max(select avg(rating)
                      from Sailor
                      group by age)
order by age;
```

Illegal SQL!
Cannot use "**where avg**(rating)=…" since **avg**(rating) is not an attribute of Sailor!

Illegal SQL!
Cannot use "**max**(…".

Is this a correct solution?

No! Why?

```
select avg(rating) as maxAvgRating, age
from Sailor
group by age
having rating=(select max(avg(rating))
               from Sailor
               group by age)
order by age;
```

Illegal SQL!
Cannot use "**having** rating=…" since rating is not in the **group by** clause.

Sailor(sailorId, sName, hkid, rating, age)

EXERCISE 5 (CONT'D)

Find the sailors for which the average rating of their age is equal to the maximum average rating of all ages.
Include in the query result tuples attributes maximum average rating and age.
Order the query result tuples by age ascending.

☞ (10, 17) (10, 30) (10, 55)

Is this a correct solution?

Yes!

```
select avg(rating) as maxAvgRating, age
from Sailor
group by age
having avg(rating)=(select max(avg(rating))
                    from Sailor
                    group by age)
order by age;
```

avg(rating)	age
7	45
3	63
10	17
10	30
8	35
1	33
10	55
1	16
5.5	25

Compute the average rating of each age.

Query result

maxAvgRating	age
10	17
10	30
10	55

Retain only the tuples where the average rating is equal to the maximum average rating of all ages.

max(avg(rating))
10

avg(rating)
7
3
10
10
8
1
10
1
5.5

Compute the average rating of each age and **select** the maximum average rating of all ages.

EXERCISE 5 (CONT'D)

Use **all**

Find the sailors for which the average rating of their age is equal to the maximum average rating of all ages.
Include in the query result tuples attributes maximum average rating and age.
Order the query result tuples by age ascending.

☞ (10, 17) (10, 30) (10, 55)

Is this a correct solution?

Yes!

```
select avg(rating) as maxAvgRating, age
from Sailor
group by age
having avg(rating)>=all(select avg(rating)
                        from Sailor
                        where rating is not null
                        group by age)
order by age;
```

avg(rating)	age
7	45
3	63
10	17
10	30
8	35
1	33
10	55
1	16
5.5	25

Compute the average rating of each age.

Query result

maxAvgRating	age
10	17
10	30
10	55

Retain only the tuples where the average rating is equal to the maximum average rating of all ages.

Can we replace >=all with <=some?

No! Why?

Will include all ages.

avg(rating)
7
3
10
10
8
1
10
1
5.5

Compute the average rating of each age.

EXERCISE 5 (CONT'D)

Use **some**

Find the sailors for which the average rating of their age is equal to the maximum average rating of all ages.
Include in the query result tuples attributes maximum average rating and age.
Order the query result tuples by age ascending.

☞ (10, 17) (10, 30) (10, 55)

Is this a correct solution?

Yes!

```
select avg(rating) as maxAvgRating, age
from Sailor
group by age
having avg(rating)=some(select max(avg(rating))
                        from Sailor
                        group by age)
order by age;
```

max(avg(rating))
10

Compute the maximum average rating of all ages.

avg(rating)	age
7	45
3	63
10	17
10	30
8	35
1	33
10	55
1	16
5.5	25

Compute the average rating of each age.

Query result

maxAvgRating	age
10	17
10	30
10	55

Retain only the tuples where the average rating is equal to the maximum average rating of all ages.

Can we use >=some?

No! Why?

Will include all ages.

Is it necessary to specify "**where** rating **is not null**" in the subquery?

No! Why?

Nulls do not participate in calculations.

EXERCISE 5 (CONT'D)

Use **fetch** clause

Find the sailors for which the average rating of their age is equal to the maximum average rating of all ages.
Include in the query result tuples attributes maximum average rating and age.
Order the query result tuples by age ascending.

Is this a correct solution?

Yes, but …

cannot order by age!

```
select avg(rating) as maxAvgRating, age
from Sailor
group by age
order by avg(rating) desc
fetch first 1 rows with ties;
```

☞ (10, 17) (10, 30) (10, 55)

avg(rating)	age
7	45
3	63
10	17
10	30
8	35
1	33
10	55
1	16
5.5	25

Compute the average rating of each age.

avg(rating)	age
10	55
10	30
10	17
8	35
7	45
5.5	25
3	63
1	33
1	16

Order by avg(rating) descending.

Query result

maxAvgRating	age
10	17
10	30
10	55

Fetch the first row with ties.

EXERCISE 5 (CONT'D)

Use **with** clause

Find the sailors for which the average rating of their age is equal to the maximum average rating of all ages.
Include in the query result tuples attributes maximum average rating and age.
Order the query result tuples by age ascending.

☞ (10, 17) (10, 30) (10, 55)

```
with temp as
    (select avg(rating) as avgRating, age
     from Sailor
     group by age)
select avgRating as maxAvgRating, age
from temp
where avgRating=(select max(avgRating)
                 from temp)
order by age;
```

temp

avg(rating)	age
7	45
3	63
10	17
10	30
8	35
1	33
10	55
1	16
5.5	25

Compute the average rating of each age.

Query result

maxAvgRating	age
10	17
10	30
10	55

Retain only the tuples where the average rating is equal to the maximum average rating in temp.

max(avgRating)
10

Compute the maximum average rating in the temp relation.

EXERCISE 5 (CONT'D)

Find the sailors for which the average rating of their age is equal to the maximum average rating of all ages.
Include in the query result tuples attributes maximum average rating and age.
Order the query result tuples by age ascending.

☞ (10, 17) (10, 30) (10, 55)

```
select avgRating as maxAvgRating, age
from (select avg(rating) as avgRating, age
      from Sailor
      group by age) temp
where avgRating=(select max(avgRating)
                 from temp)
order by age;
```

- This query is correct SQL but will raise an error in Oracle.
  - `ORA-00942: table or view does not exist`

☞ **Oracle restricts the scope of the alias temp to the outer select.**

EXERCISE 6

Find the number of reservations made for any three sailors who have made the highest number of reservations.
Include in the query result tuples attributes sailor name and number of reservations.
Order the query result tuples by number of reservations descending.

☞ (Dustin, 4) (Lucy, 3) (Horatio, 2)

```
select sName, count(sName) numReservations
from Sailor natural join Reserves
group by sailorId, sName
order by numReservations desc
fetch first 3 rows with ties;
```

Is sailorId needed in the **group by** clause? Yes! Why?

sName is not unique. Therefore, the count may be wrong.

```
select sName, count(sName) numReservations
from Sailor natural join Reserves
group by sName
order by numReservations desc
fetch first 3 rows with ties;
```

Query result

sName	numReservations
Dustin	4
Lucy	3
Horatio	3

X

Sailor(sailorId, sName, hkid, rating, age)

Reserves(*sailorId, boatId,* rDate)

EXERCISE 7

Find ratings having at least two adult sailors (i.e., age≥18) with the rating. Include in the query result tuples attributes rating, the number of sailors with the rating and the age of the youngest sailor with the rating. Order the query result tuples by rating ascending.

☞ (3, 2, 25) (7, 2, 35) (10, 3, 17)

EXERCISE 7 (CONT'D)

☞ (3, 2, 25) (7, 2, 35) (10, 3, 17)

Is this a correct solution?

No!

Why?

```
select rating, count(*) as numSailors, min(age)
from Sailor
where age>=18
group by rating
having count(*)>=2
order by rating;
```

Project on rating, count and minimum age for each selected group.

Get the sailors whose age is greater than or equal to 18.

sailorId	sName	hkid	rating	age
~~10~~	~~Emily~~	~~P936219(3)~~	~~null~~	~~16~~
~~15~~	~~Zoey~~	~~K091876(5)~~	~~null~~	~~17~~
22	Dustin	A298456(6)	7	45
29	Brutus	D746239(0)	1	33
31	Lucy	P930281(5)	10	55
32	Andy	M028391(4)	8	25
~~58~~	~~Rachael~~	~~A819204(3)~~	~~10~~	~~17~~
64	Horatio	F710943(2)	7	35
~~71~~	~~Zorba~~	~~P910562(4)~~	~~1~~	~~16~~
74	Horatio	T810463(1)	9	35
85	Amy	B719037(8)	3	25
95	Bob	E012735(5)	3	63
99	Chris	H814243(7)	10	30

Group the result by rating.

sailorId	sName	hkid	rating	age
29	Brutus	D746239(0)	1	33
85	Amy	B719037(8)	3	25
95	Bob	B719037(8)	3	63
22	Dustin	A298456(6)	7	45
64	Horatio	F710943(2)	7	35
32	Andy	M028391(4)	8	25
74	Horatio	T810463(1)	9	35
31	Lucy	P930281(5)	10	55
99	Chris	A819204(3)	10	30

✓ ✓ ✓

Retain only those groups that have at least 2 sailors.

Query result

rating	numSailors	min(age)
3	2	25
7	2	35
10	2	30

EXERCISE 7 (CONT'D)

Find ratings having at least two adult sailors (i.e., age≥18) with the rating. Include in the query result tuples attributes rating, the number of sailors with the rating and the age of the youngest sailor with the rating. Order the query result tuples by rating ascending.

☞ (3, 2, 25) (7, 2, 35) (10, 3, 17)

```
select rating, count(*) as numSailors, min(age) as minAge
from Sailor
where age>=18
group by rating
having rating in (select rating
                  from Sailor
                  where age>=18
                  group by rating
                  having count(*)>=2)
order by rating;
```

rating
3
7
10

The ratings for which there are at least 2 adult sailors.